Practica 1
Algoritmos de Ordenamiento

Ordenamiento alfabetico de un dataset de 100.000 palabras en C++

Estructura de Datos
Implementacion desde cero de: Burbuja, Insercion, Quicksort y Mergesort
Medicion de tiempos con la libreria `<chrono>`

1. Introduccion

El objetivo de esta practica es implementar **desde cero** cuatro algoritmos de ordenamiento (burbuja, insercion, quicksort y mergesort) para ordenar alfabeticamente un dataset largo de palabras, sin utilizar ninguna funcion de libreria que ya realice el ordenamiento (por ejemplo `std::sort`). Como cuarto algoritmo de libre eleccion se escogio **mergesort**.

Las palabras se almacenan en un arreglo dinamico `std::vector<std::string>`, se cargan desde el archivo `dataset.txt` (100.000 palabras), y para cada algoritmo se mide el tiempo de ejecucion con la libreria `<chrono>`. Finalmente se comparan los tiempos y se redactan conclusiones.

> **Nota sobre el dataset:** el archivo original venia codificado en UTF-16. Se convirtio a ASCII/UTF-8 para que la lectura con `ifstream >>` en C++ fuera limpia, sin caracteres de control intercalados entre las letras.

2. Codigo comun a los cuatro programas

Los cuatro programas comparten la misma estructura de entrada/salida. A continuacion se explican esas partes comunes una sola vez para no repetirlas en cada algoritmo.

2.1. Librerias incluidas

```
#include <iostream>   // cout, cerr -> imprimir por consola
#include <fstream>    // ifstream, ofstream -> leer/escribir archivos
#include <vector>     // vector -> array dinamico para las palabras
#include <string>     // string -> tipo de dato de cada palabra
#include <chrono>     // chrono -> medicion precisa del tiempo
```

> **<iostream>:** permite escribir mensajes al usuario (cantidad de palabras, tiempo).

> **<fstream>:** permite leer el dataset del disco y guardar el resultado ordenado.

> **<vector>:** aporta el array dinamico que crece automaticamente segun cuantas palabras haya.

> **<string>:** tipo que representa cada palabra y permite compararlas con < y >.

> **<chrono>:** reloj de alta resolucion para medir el tiempo de ordenamiento.

2.2. Funcion cargarPalabras

```cpp
std::vector<std::string> cargarPalabras(const std::string& nombreArchivo) {
    std::vector<std::string> palabras;          // vector vacio
    std::ifstream archivo(nombreArchivo);       // abre en modo lectura

    if (!archivo.is_open()) {                   // valida que exista
        std::cerr << "No se pudo abrir el archivo: " << nombreArchivo << "\n";
        return palabras;                        // devuelve vacio si falla
    }

    std::string palabra;                        // temporal por palabra
    while (archivo >> palabra) {                // lee token a token hasta EOF
        palabras.push_back(palabra);            // agrega al vector
    }

    archivo.close();                            // cierra el archivo
    return palabras;                            // devuelve el vector lleno
}
```

> **std::vector<std::string> palabras:** se declara el contenedor donde se guardaran todas las palabras. Se usa porque no sabemos de antemano cuantas hay.

> **std::ifstream archivo(nombreArchivo):** abre el archivo de texto para lectura. Es la forma estandar de leer archivos en C++.

> **if (!archivo.is_open()):** comprueba que el archivo realmente se abrio; si no, avisa por `cerr` y evita continuar con datos vacios.

> **while (archivo >> palabra):** el operador `>>` lee una palabra a la vez saltando espacios y saltos de linea; el bucle termina al llegar al fin del archivo.

> **palabras.push_back(palabra):** agrega cada palabra leida al final del vector, que crece dinamicamente segun se necesite.

> **archivo.close():** libera el archivo. Es buena practica cerrarlo cuando ya no se usa.

2.3. Funcion guardarPalabras

```cpp
void guardarPalabras(const std::string& nombreArchivo,
                     const std::vector<std::string>& palabras) {
    std::ofstream salida(nombreArchivo);        // abre/crea para escritura
    for (const std::string& p : palabras) {     // recorre cada palabra
        salida << p << "\n";                    // la escribe con salto de linea
    }
    salida.close();                             // cierra el archivo
}
```

> **std::ofstream salida(nombreArchivo):** abre (o crea) el archivo de salida donde se guardara el resultado ya ordenado.

> **for (const std::string& p : palabras):** recorre el vector por referencia (`&`) para no copiar cada palabra, lo que es mas eficiente.

> **salida << p << "\n":** escribe la palabra seguida de un salto de linea, dejando una palabra por linea en el archivo final.

2.4. Medicion del tiempo (comun en main)

```
auto inicio = std::chrono::high_resolution_clock::now();  // t antes
// ... aqui se llama al algoritmo de ordenamiento ...
auto fin = std::chrono::high_resolution_clock::now();     // t despues
std::chrono::duration<double, std::milli> duracion = fin - inicio;
std::cout << "Tiempo de ejecucion: " << duracion.count() << " ms\n";
```

> **high_resolution_clock::now():** captura el instante actual con la mayor precision disponible, justo antes y despues de ordenar.

> **fin - inicio:** la resta de dos instantes produce una duracion.

> **duration<double, std::milli>:** expresa esa duracion en milisegundos con decimales, para poder comparar algoritmos rapidos y lentos en la misma unidad.

> **duracion.count():** extrae el numero para imprimirlo.

3. Algoritmo Burbuja (Bubble Sort)

La burbuja recorre el vector repetidamente comparando **pares de elementos adyacentes**. Si estan en el orden incorrecto los intercambia, haciendo que el mayor "burbujee" hacia el final en cada pasada. Su complejidad es $O(n^2)$.

```
void bubbleSort(std::vector<std::string>& v) {
    int n = v.size();                          // numero de elementos

    for (int i = 0; i < n - 1; i++) {          // cuantas pasadas
        bool huboIntercambio = false;          // bandera de optimizacion

        for (int j = 0; j < n - 1 - i; j++) {  // pares adyacentes
            if (v[j] > v[j + 1]) {             // compara alfabeticamente
                std::string temp = v[j];       // guarda temporal
                v[j] = v[j + 1];               // menor a la izquierda
                v[j + 1] = temp;               // mayor a la derecha
                huboIntercambio = true;        // hubo cambio
            }
        }
        if (!huboIntercambio) break;           // ya esta ordenado
    }
}
```

> **int n = v.size():** guarda cuantos elementos hay para controlar los bucles.

> **for (i = 0; i < n-1; i++):** bucle externo; controla el numero de pasadas. Con n-1 pasadas basta para ordenar todo.

> **bool huboIntercambio = false:** bandera que optimiza: si en una pasada no hubo cambios, el arreglo ya esta ordenado.

> **for (j = 0; j < n-1-i; j++):** bucle interno; se resta `i` porque tras cada pasada el mayor ya quedo fijo al final y no hace falta revisarlo.

> **if (v[j] > v[j+1]):** compara dos palabras vecinas; el operador `>` sobre `string` compara alfabeticamente (orden lexicografico).

> **temp / v[j] / v[j+1]:** las tres lineas realizan el intercambio (swap) usando una variable temporal para no perder ningun valor.

> **huboIntercambio = true:** marca que en esta pasada si se movio algo.

> **if (!huboIntercambio) break:** si una pasada completa no hizo intercambios, se corta el bucle: el arreglo ya esta ordenado.

4. Algoritmo Insercion (Insertion Sort)

Construye el arreglo ordenado un elemento a la vez: toma cada elemento y lo **inserta** en su posicion correcta dentro de la parte ya ordenada a su izquierda, desplazando los mayores una posicion a la derecha. Complejidad $O(n^2)$, pero es rapido si los datos ya estan casi ordenados.

```cpp
void insertionSort(std::vector<std::string>& v) {
    int n = v.size();                           // numero de elementos

    for (int i = 1; i < n; i++) {               // desde el 2do elemento
        std::string clave = v[i];               // elemento a insertar
        int j = i - 1;                          // ultimo ya ordenado

        while (j >= 0 && v[j] > clave) {        // mayores a la izquierda
            v[j + 1] = v[j];                    // desplaza a la derecha
            j--;                                // sigue hacia la izquierda
        }
        v[j + 1] = clave;                       // coloca la clave en su hueco
    }
}
```

> **for (i = 1; i < n; i++):** empieza en 1 porque el primer elemento se considera, por si solo, ya ordenado.

> **std::string clave = v[i]:** guarda el elemento actual que se va a insertar en la parte ordenada.

> **int j = i - 1:** apunta al ultimo elemento de la zona ya ordenada, para comparar hacia atras.

> **while (j >= 0 && v[j] > clave):** mientras existan elementos a la izquierda que sean mayores que la clave, hay que hacerles espacio.

> **v[j+1] = v[j]:** desplaza el elemento mayor una posicion a la derecha, abriendo hueco.

> **j--:** retrocede para seguir comparando con el siguiente elemento hacia la izquierda.

> **v[j+1] = clave:** cuando ya no hay mayores (o se llego al inicio), coloca la clave en el hueco correcto.

5. Algoritmo Quicksort

Quicksort usa la estrategia **divide y venceras**. Escoge un elemento como pivote, reorganiza el arreglo de modo que los menores queden a la izquierda y los mayores a la derecha (particion), y luego ordena recursivamente cada mitad. Complejidad promedio O(n log n). Aqui se usa el esquema de particion de **Lomuto** con el ultimo elemento como pivote.

5.1. Funcion particion

```
int particion(std::vector<std::string>& v, int bajo, int alto) {
    std::string pivote = v[alto];               // pivote = ultimo
    int i = bajo - 1;                           // frontera de menores

    for (int j = bajo; j < alto; j++) {         // recorre el rango
        if (v[j] < pivote) {                    // si es menor al pivote
            i++;                                // avanza la frontera
            std::string temp = v[i];            // swap v[i] <-> v[j]
            v[i] = v[j];
            v[j] = temp;
        }
    }
    std::string temp = v[i + 1];                // coloca el pivote
    v[i + 1] = v[alto];                         // en su posicion final
    v[alto] = temp;
    return i + 1;                               // posicion del pivote
}
```

> **std::string pivote = v[alto];:** elige como pivote el ultimo elemento del rango; sera el valor de referencia para dividir.

> **int i = bajo - 1:** marca la frontera entre "los menores encontrados" y el resto; empieza antes del rango.

> **for (j = bajo; j < alto; j++):** recorre todos los elementos excepto el pivote.

> **if (v[j] < pivote):** si el elemento actual es menor que el pivote, pertenece al grupo izquierdo.

> **i++ y el swap:** se amplia la zona de menores y se intercambia el elemento actual hacia esa zona.

> **los tres ultimos swaps:** colocan el pivote justo despues del ultimo menor (posicion `i+1`), su lugar definitivo.

> **return i + 1:** devuelve la posicion final del pivote, que divide el arreglo en dos mitades a ordenar.

5.2. Funcion quickSort (recursiva)

```
void quickSort(std::vector<std::string>& v, int bajo, int alto) {
    if (bajo < alto) {                          // caso base: >1 elemento
        int p = particion(v, bajo, alto);       // ubica el pivote
        quickSort(v, bajo, p - 1);              // ordena mitad izquierda
        quickSort(v, p + 1, alto);              // ordena mitad derecha
    }
}
```

> **if (bajo < alto):** condicion de parada de la recursion; si el rango tiene 0 o 1 elementos ya esta ordenado.

> **int p = particion(...):** particiona el rango y obtiene la posicion final del pivote.

> **quickSort(v, bajo, p-1):** ordena recursivamente la parte de la izquierda (los menores).

> **quickSort(v, p+1, alto):** ordena recursivamente la parte de la derecha (los mayores). El pivote ya quedo en su lugar.

6. Algoritmo Mergesort (cuarto algoritmo elegido)

Mergesort tambien usa **divide y venceras**: parte el arreglo por la mitad, ordena cada mitad de forma recursiva y luego **mezcla** las dos mitades ordenadas en una sola. Su complejidad es O(n log n) **garantizada** en todos los casos, a costa de usar memoria adicional para las mezclas.

6.1. Funcion merge (mezcla)

```
void merge(std::vector<std::string>& v, int izq, int medio, int der) {
    int n1 = medio - izq + 1;                    // tam. mitad izquierda
    int n2 = der - medio;                        // tam. mitad derecha

    std::vector<std::string> L(n1), R(n2);        // vectores temporales
    for (int i = 0; i < n1; i++) L[i] = v[izq + i];
    for (int j = 0; j < n2; j++) R[j] = v[medio + 1 + j];

    int i = 0, j = 0, k = izq;                   // indices L, R y escritura
    while (i < n1 && j < n2) {                   // mezcla comparando frentes
        if (L[i] <= R[j]) v[k++] = L[i++];
        else              v[k++] = R[j++];
    }
    while (i < n1) v[k++] = L[i++];             // resto de L
    while (j < n2) v[k++] = R[j++];             // resto de R
}
```

> **n1 y n2:** calculan el tamano de cada mitad a mezclar.

> **L(n1), R(n2):** vectores temporales que reciben una copia de cada mitad; son necesarios porque la mezcla se escribe encima del original.

> **los dos for de copia:** llenan L con la mitad izquierda y R con la derecha.

> **int i=0, j=0, k=izq:** `i` recorre L, `j` recorre R y `k` es la posicion donde se escribe en el vector original.

> **while (i<n1 && j<n2):** compara el frente de cada mitad y copia el menor al arreglo, avanzando ese indice. Asi la mezcla queda ordenada.

> **if (L[i] <= R[j]):** usa `<=` para mantener estabilidad (si son iguales, entra primero el de la izquierda).

> **los dos while finales:** copian los elementos que hayan quedado en una mitad cuando la otra se agoto.

6.2. Funcion mergeSort (recursiva)

```
void mergeSort(std::vector<std::string>& v, int izq, int der) {
    if (izq < der) {                             // caso base: >1 elemento
        int medio = izq + (der - izq) / 2;       // punto medio
        mergeSort(v, izq, medio);                // ordena izquierda
        mergeSort(v, medio + 1, der);            // ordena derecha
        merge(v, izq, medio, der);               // mezcla ambas
    }
}
```

> **if (izq < der):** condicion de parada; un rango de 0 o 1 elementos ya esta ordenado.

> **int medio = izq + (der - izq) / 2:** calcula el punto medio de esta forma para evitar desbordamiento (overflow) al sumar indices grandes.

> **mergeSort(v, izq, medio):** ordena recursivamente la mitad izquierda.

> **mergeSort(v, medio+1, der):** ordena recursivamente la mitad derecha.

> **merge(v, izq, medio, der):** combina las dos mitades ya ordenadas en un solo segmento ordenado.

7. Comparacion de tiempos de ejecucion

Todos los algoritmos ordenaron el mismo dataset de **100.000 palabras**. Se verifico que las cuatro salidas son **identicas byte a byte**, lo que confirma que las cuatro implementaciones ordenan correctamente. Los programas se compilaron con `g++ -O2` (MinGW-w64 16.1.0).

Algoritmo	Complejidad	Tiempo medido	Relativo a Quicksort
Quicksort	O(n log n) promedio	**41,81 ms**	1x (mas rapido)
Mergesort	O(n log n) garantizado	61,42 ms	~1,5x
Insercion	$O(n^2)$	30.955 ms (~31 s)	~740x
Burbuja	$O(n^2)$	**103.824 ms (~1 min 44 s)**	~2.483x mas lento

7.1. Grafico comparativo (escala logaritmica)

El eje se muestra en escala logaritmica porque la diferencia es de varios ordenes de magnitud.

Quicksort		41,81 ms
Mergesort		61,42 ms
Insercion		30.955 ms
Burbuja		103.824 ms

8. Conclusiones

- **El algoritmo importa mas que la maquina.** Sobre el mismo dataset y hardware, quicksort fue unas **2.483 veces** mas rapido que burbuja. La eleccion del algoritmo tiene un impacto enorme cuando los datos son grandes.
- **O(n log n) vence claramente a $O(n^2)$ a gran escala.** Con 100.000 palabras, quicksort y mergesort terminan en milisegundos, mientras que burbuja e insercion tardan decenas de segundos o minutos. La diferencia crece a medida que el dataset se agranda.
- **Insercion supera a burbuja** aunque ambos sean $O(n^2)$, porque insercion realiza desplazamientos en lugar de multiples intercambios completos y puede cortar antes. Fue ~3,4 veces mas rapido que burbuja.
- **Quicksort fue el mas rapido** en la practica gracias a su buen aprovechamiento de la memoria cache y a que no necesita arreglos auxiliares. Su desventaja teorica es el peor caso $O(n^2)$, que no se presento con estos datos.
- **Mergesort es la opcion mas predecible:** garantiza O(n log n) siempre y es estable, a cambio de usar memoria adicional para las mezclas. Quedo muy cerca de quicksort.
- **Burbuja e insercion siguen teniendo valor didactico** y son utiles para listas muy pequenas o casi ordenadas, pero no son adecuados para datasets grandes como este.